Universität Rostock

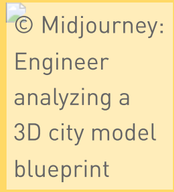
# GEOMETRIEPROBLEME

Joern Ploennigs · AI4SC

*Midjourney: Geometric city plan viewed from above, architectural blueprint*

# Geometrische Probleme treten im Bauwesen in vielen Aufgaben auf

Wo habt ihr bereits mit Koordinaten, Vektoren oder Polygonen gearbeitet?



© Midjourney: Engineer analyzing a 3D city model blueprint

# GEOMETRISCHE PROBLEME

# Was macht ein Problem geometrisch?

⇆

Ein typischer Fehler aus der Praxis: Eine Leitung wirkt im Modell kollisionsfrei oder ein Gebäude scheint “nah genug”, obwohl Distanz oder Lage im falschen Koordinatensystem berechnet wurden.

**Geometrische Probleme** drehen sich um:

- Räumliche Objekte: Punkte, Linien, Flächen
- ⇆ Räumliche Beziehungen: enthält, schneidet, nächster
- Koordinaten, Maße, Orientierungen

**Problemklassen-Signatur:**

- *Eingabe:* Koordinaten, Formen, Abstände
- *Frage:* Liegt A in B? Wie weit? Überlappen sie?
- *Ausgabe:* Bool, Zahl oder neue Geometrie

⇆ Problemklasse

**Geometrische Domänen:**

| Domäne | Beispiel |
|---|---|
| CAD | Gebäudegrundriss prüfen |
| BIM | Bauteilkollision erkennen |
| GIS | Gebäude in Überschwemmungszone |
| Netze | Trassierung einer Straße |

# Typische Daten und Fragen im Bauwesen

## CAD

- Daten: Punkte, Linien, Polygone
- Frage: Passt ein Bauteil in den Grundriss?
- Ziel: Geometrie korrekt konstruieren

## BIM

- Daten: Bauteile mit Lage und Ausdehnung
- Frage: Schneiden sich zwei Elemente?
- Ziel: Kollisionen und Mengen prüfen

## GIS

- Daten: Flächen, Netze, Zonen
- Frage: Liegt ein Objekt in einer Risikozone?
- Ziel: räumliche Analyse und Bewertung

# INGENIEURSANWENDUNGEN

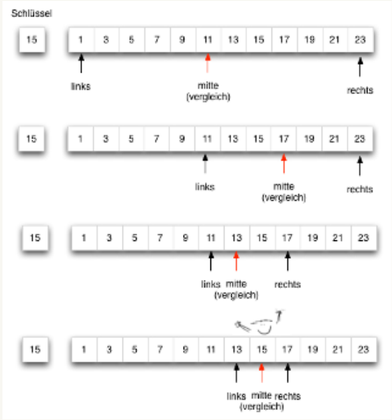
# Geometrie im Bauwesen — Beispiele

**Gebäude und Infrastruktur:**

- Grundfläche eines Gebäudes berechnen
- Sensor innerhalb eines Raums prüfen
- Rohr mit Decke auf Kollision prüfen
- Straßenachse als Linienzug modellieren

**Raumbezogene Analyse:**

- Gebäude in Überschwemmungszone ermitteln
- Nächste Haltestelle bestimmen
- Kreuzung unterirdischer Leitungen erkennen




Midjourney: Gebäudekontur als abstrahierter Lageplan

# Gemeinsame Problemstruktur

**Problem 1:** Sensor im Raum? → *Point-in-Polygon*

**Problem 2:** Nächstes Krankenhaus? → *Distanzberechnung*

**Problem 3:** Rohrkollision? → *Schnittprüfung*

**Problem 4:** Überlappen zwei Bauparzellen? → *Flächenschnitt*

Muster erkennen

Alle vier Fragen nutzen nur drei Grundoperationen:

01 **contains / within** — Enthaltensein

02 **distance** — Abstand

03 **intersects / intersection** — Schnitt

Dieselbe Bibliothek löst alle vier.

# VEKTORALGEBRA

# Geometrische Grundobjekte

Datenstrukturen

**Punkt:** `(x, y)` oder `(x, y, z)`

**Liniensegment:** Liste von 2+ Punkten

**Polygon:** Geordnete Punktliste, konzeptuell geschlossen

In vielen Datenformaten wird der erste Punkt am Ende wiederholt. In `shapely` darf die Schlusskante meist weggelassen werden, weil `Polygon(...)` sie automatisch ergänzt.

```python
# Punkt
p = (3.0, 2.5)

# Liniensegment
linie = [(0, 0), (10, 5)]

# Polygon (Rechteck)
polygon = [
    (0, 0),
    (10, 0),
    (10, 5),
    (0, 5)
]

# shapely schließt das Polygon automatisch
```

# Ein Beispiel von Punkt zu Operation

**Mini-Beispiel:**

- Raum als Rechteck modellieren
- Sensor als Punkt modellieren
- Danach erst die Frage “innen oder außen?” stellen

```
raum = [(0, 0), (10, 0), (10, 5), (0, 5)]
sensor = (3, 2)

# Noch keine Antwort auf die Frage:
# Wir haben nur die Geometrie modelliert.
```

So trennen wir Repräsentation und Operation:

01 Objekt als Datenstruktur beschreiben

02 Passende geometrische Operation wählen

# Vektor-Operationen

Algorithmen

**Abstand zwischen zwei Punkten:**

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

**Kreuzprodukt (Orientierung):**

$$\vec{u} \times \vec{v} = u_x v_y - u_y v_x$$

- Positiv → Punkt liegt links der Linie
- Negativ → Punkt liegt rechts der Linie
- Null → Punkt liegt auf der Linie

**Anwendung des Kreuzprodukts:**

- Grundlage für Point-in-Polygon
- Orientierung von Polygonecken
- Kollisionserkennung

```
def kreuzprodukt_2d(u, v):
    return u[0]*v[1] - u[1]*v[0]

# Liegt (3,2) links von (0,0)→(5,5)?
u = (5-0, 5-0)  # Richtung
v = (3-0, 2-0)  # zum Punkt
k = kreuzprodukt_2d(u, v)
# k > 0 → links
```

# GRUNDOPERATIONEN

# Point-in-Polygon: Liegt ein Punkt in einer Zone?

**Problem:** Liegt Sensor in Gebäude? Liegt Adresse in Überschwemmungszone?

**Ray-Casting-Algorithmus:**

01 Strahl vom Punkt nach rechts

02 Zähle Schnitte mit Polygon-Kanten

03 Ungerade Anzahl → Punkt liegt innen

**Merksatz**

Ungerade = drinnen. Gerade = draußen.

```python
from shapely.geometry import Point, Polygon

zone = Polygon([
    (0,0),(10,0),(10,5),(0,5)
])
sensor = Point(3, 2)

# contains: Punkt vollständig innen?
print(zone.contains(sensor))   # True
print(zone.contains(Point(15, 2)))  # False

# within ist die Umkehrung
print(sensor.within(zone))     # True
```

# Distanz und Schnitt

**Distanz** — kürzester Abstand zweier Geometrien:

```python
from shapely.geometry import Point

krankenhaus = Point(5, 5)
unfallort   = Point(12, 8)

dist = krankenhaus.distance(unfallort)
print(f"{dist:.2f}")  # 7.62
```

**Anwendung:** Nächste Haltestelle, nächstes Krankenhaus, nächste Feuerwehr

**Schnitt** — überlappen sich zwei Geometrien?

```python
from shapely.geometry import LineString

rohr1 = LineString([(0,0),(10,10)])
rohr2 = LineString([(0,10),(10,0)])

# Kreuzen sie sich?
print(rohr1.intersects(rohr2))  # True

# Wo genau?
punkt = rohr1.intersection(rohr2)
print(punkt)  # POINT (5 5)
```

**Anwendung:** Kollisionsprüfung in BIM

# SHAPELY

# shapely — Die geometrische Standardbibliothek

Bibliothek

**Kerntypen:**

- `Point` — Koordinate
- `LineString` — Linienzug
- `Polygon` — Fläche mit Grenze
- `MultiPolygon` — z.B. Parzellengruppe

**Kernoperationen:**

- `.contains()`, `.within()`
- `.intersects()`, `.intersection()`
- `.distance()`, `.area`, `.length`
- `.buffer()` — Pufferzone erzeugen

```python
from shapely.geometry import Point, Polygon

gebaeude = Polygon([
    (0, 0), (20, 0),
    (20, 15), (0, 15)
])

print(f"Fläche: {gebaeude.area} m²")
# Fläche: 300 m²

# 5m Pufferzone
puffer = gebaeude.buffer(5)
print(f"Mit Puffer: {puffer.area:.0f} m²")
# Mit Puffer: 729 m²

# Enthält ein Punkt die Zone?
sensor = Point(10, 7)
```

# KOORDINATENSYSTEME

# Warum Projektionen wichtig sind

⇆

**Problem:** Die Erde ist eine Kugel — Karten sind flach.

**WGS84 (EPSG:4326):**

- Koordinaten in Grad (Länge/Breite)
- Standard für GPS, Google Maps, GeoJSON
- 1 Grad ≠ 1 Meter!

**Lokale Systeme (z.B. UTM, EPSG:25833):**

- Koordinaten in Metern
- Geeignet für lokale Distanzberechnungen
- Verzerrt über große Gebiete

> **Häufiger KI-Fehler**
>
> KI-generierter Code berechnet Abstände direkt aus WGS84-Koordinaten. `shapely` kennt kein KBS — `distance()` liefert Werte in den Einheiten der Koordinaten, also hier in Grad und nicht automatisch in Metern.

# Projektionen im Code

**Falsch — typischer KI-Output:**

```python
from shapely.geometry import Point

# Koordinaten in WGS84 (Grad)
berlin  = Point(13.405, 52.520)
potsdam = Point(13.064, 52.400)

dist = berlin.distance(potsdam)
# ≠ Meter! Einheit: Grad
print(dist)  # 0.35 — bedeutungslos
```

**Problem:** 0.35 Grad ≈ 35 km, aber `shapely` weiß das nicht.

**Richtig — mit Projektion:**

```python
from pyproj import Transformer
from shapely.ops import transform

# WGS84 → UTM Zone 33N
t = Transformer.from_crs(4326, 25833,
                         always_xy=True)

berlin_utm  = transform(t.transform, berlin)
potsdam_utm = transform(t.transform, potsdam)

dist_m = berlin_utm.distance(potsdam_utm)
print(f"{dist_m:.0f} m")  # 28 422 m ✓
```

# KI-Code mit shapely lesen — Aufgabe

KI-Code lesen

```python
# KI-generiert: Gebäude im Radius finden
from shapely.geometry import Point, Polygon

def gebaeude_in_radius(gebaeude_liste, mittelpunkt_coords, radius_m):
    mittelpunkt = Point(mittelpunkt_coords)
    suchzone = mittelpunkt.buffer(radius_m)
    treffer = []
    for geb in gebaeude_liste:
        polygon = Polygon(geb['koordinaten'])
        if suchzone.intersects(polygon):
            treffer.append(geb)
    return treffer
```

⚠ **Versteht den Code — nach dem CRS-Block**

01 `buffer(radius_m)` erzeugt eine Suchzone in den Einheiten der Eingabekoordinaten.

02 `intersects()` genügt hier, weil auch teilweise überdeckte Gebäude gezählt werden sollen.

03 Bei WGS84 in Grad fehlt vor dem `buffer()` eine Projektion in ein metrisches KBS.

# KI-PROMPT-MUSTER

# Prompt-Muster für geometrische Probleme

Agentic Cycle

**Schlechter Prompt:**

> “Schreibe Python-Code, um Gebäude in der Nähe eines Punktes zu finden.”

**Fehlende Angaben:**

- Kein Koordinatensystem
- Kein Datenformat
- Keine Einheit für “Nähe”
- Keine Bibliotheksvorgabe

**Guter Prompt:**

> “Gegeben: Gebäudepolygone als Liste von WGS84-Koordinaten (Länge, Breite). Aufgabe: Alle Gebäude zurückgeben, die innerhalb von 500 m eines gegebenen Punktes liegen. Verwende `shapely` und `pyproj`. Transformiere alle Geometrien zuerst nach UTM Zone 33N (EPSG:25833).”

**Checkliste Prompt Geometrie:**

- Koordinatensystem angeben
- Einheit angeben (Meter, Grad)
- Bibliothek benennen ( `shapely` )

# KI-Fehler bei Geometrie erkennen

**Häufige KI-Fehler:**

- Kein KBS angegeben oder verwendet
- `distance()` auf WGS84-Koordinaten (Grad)
- Bibliotheksverhalten zu Polygonringen falsch angenommen
- Koordinatenreihenfolge vertauscht (lat/lon statt lon/lat)
- Gleichheitsvergleich ohne Fließkomma-Toleranz

**Prüffragen beim Code-Review:**

- Eingabe-KBS bestimmen
- Einheit der Distanzberechnung prüfen
- Polygonformat für die verwendete Bibliothek prüfen

```python
# Fließkomma-Toleranz

from shapely.geometry import Point, LineString

p    = Point(1.0, 2.0)
linie = LineString([(0,0),(2,4)])

# Falsch: exakter Vergleich
if linie.distance(p) == 0:
    print("auf der Linie")

# Richtig: Toleranz
TOL = 1e-9
if linie.distance(p) < TOL:
    print("auf der Linie")
```

# VERIFIKATION

# Verifikations-Checkliste

Agentic Cycle · Geometrie

**Vor dem Ausführen:**

- Geometrietyp bestimmen: Punkt, Linie oder Polygon
- KBS prüfen: WGS84 oder metrisches System
- Polygonformat für die Bibliothek prüfen
- Koordinatenreihenfolge prüfen: `(lon, lat)` oder `(lat, lon)`

**Beim Testen:**

- Mit bekannter Geometrie testen: Einheitsquadrat
- Grenzfälle prüfen: Punkt auf Kante, Punkt im Eckpunkt
- Numerische Ergebnisse mit Handrechnung vergleichen
- Distanzwerte auf Plausibilität prüfen

**Agentic workflow:** Bei KI-generiertem Geometriecode:

01 KBS prüfen — steht es im Code oder fehlt es?

02 Geometrietyp prüfen — `.geom_type` ausgeben lassen

03 Einfachen Testfall ausführen (Einheitsquadrat: Fläche = 1?)

04 Distanzberechnung prüfen — Einheit plausibel?

05 Fließkomma-Toleranz ergänzen wenn nötig

# QUESTIONS